

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0603

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning (**BL 5**)

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks:20

Annexure A

Comparative Analysis

Code of Conduct:

I X. Joshua – (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

X. Joshua

1. KNN vs Case-Based Learning — Medical Diagnosis

Both **KNN** and **Case-Based Learning (CBL)** can diagnose a new patient by comparing the patient with previous cases.

Factor	KNN	Case-Based Learning
Memory	Stores training examples	Stores complete past cases
Similarity	Usually Euclidean/Cosine distance	Domain-specific similarity measures
Prediction	Majority class of K neighbours	Solution from most similar case(s)
Adaptation	Add new training samples	New cases can be directly stored
Explanation	Shows nearest neighbours	Can explain using previous case

Example:

For a patient with fever, cough, age, and blood pressure, KNN finds the K most similar patients and predicts the majority diagnosis. CBL retrieves the most similar previous patient case and reuses its diagnosis.

Conclusion: KNN is simple and effective for classification, while CBL is better when previous complete cases and their solutions are valuable for explanation and reuse.

2. LWR vs RBF Networks — Function Approximation

Suppose we want to approximate:

using sample points.

Factor	LWR	RBF
Locality	Builds local model for each query	Uses fixed Gaussian basis functions
Training	Little/no global training	Learns centers and output weights
Prediction cost	Can be high for every query	Usually faster after training
Smoothness	Depends on bandwidth	Usually smooth
Adaptability	Very high	Requires retraining/update

LWR: Nearby points receive higher weights, so prediction focuses on the local region.

RBF: Gaussian functions centered at selected points create smooth non-linear approximation.

Conclusion: LWR is useful when local behaviour changes significantly, while RBF is preferable when many predictions are required after a training phase.

3. KNN vs RBF — Classification

Consider classifying emails as **Spam** or **Not Spam**.

Factor	KNN	RBF
Decision boundary	Based on neighbours	Learned using Gaussian functions
Training time	Very low	Higher
Prediction	Can be expensive	Usually faster
Generalization	Depends strongly on K/data	Can generalize smoothly
Model	Instance-based	Trained model

KNN: For a new email, it finds nearby training emails and uses majority voting.

RBF: Input features are transformed through Gaussian basis functions and an output layer performs classification.

Decision boundary: KNN boundaries can be irregular and depend on training samples. RBF can create smoother non-linear boundaries.

Conclusion: KNN is simple and useful for small datasets, while RBF is generally better when repeated predictions and non-linear boundaries are required.

4. Case-Based Learning vs LWR — Recommendation System

Suppose an online store recommends products.

Factor	CBL	LWR
Main purpose	Reuse previous cases	Predict continuous output
New data	Store as new case	Influences local regression
Similarity	Direct case similarity	Distance determines weights
Adaptability	Very high	High
Output	Previous/reused solution	Numerical prediction

CBL: A new customer is compared with previous customers. The system retrieves a similar customer and reuses their successful recommendation.

LWR: Customer features such as spending, frequency, and product interest can be used to predict a numerical recommendation score using locally weighted data.

Conclusion: CBL is better when previous recommendations can be directly reused. LWR is better when recommendation scores or continuous values need to be predicted.

5. Decision Tree vs KNN — Customer Churn

Factor	Decision Tree	KNN
Interpretability	Very high	Lower
Training time	Moderate	Very low
Prediction	Fast	Can be slower
Noise	Can overfit noise	Sensitive to noisy neighbours
Large datasets	Generally efficient	Prediction becomes expensive
Decision	If-then rules	Similarity/majority voting

Example:

Features can include contract type, monthly bill, usage, and customer age.

A Decision Tree might produce:

If contract = monthly AND bill > ₹1000 → High churn.

KNN finds customers with similar characteristics and uses their churn status.

Conclusion: Decision Trees are preferable when interpretability is important. KNN is useful for smaller datasets where similarity between customers is meaningful.

6. Naïve Bayes vs Logistic Regression — Spam Detection

Factor	Naïve Bayes	Logistic Regression
Assumption	Features conditionally independent	No independence assumption
Output	Probability	Probability
Training	Very fast	Fast
High-dimensional text	Excellent	Excellent
Decision boundary	Probabilistic	Linear boundary
Data requirement	Works well with less data	Often benefits from more data

Naïve Bayes: Calculates probability that an email is spam based on words such as "offer", "free", and "winner".

Logistic Regression: Learns weights for words and calculates the probability of spam.

When NB is better: Small datasets and very high-dimensional sparse text.

When Logistic Regression is better: When relationships between features are important and sufficient labelled data is available.

7. SVM vs KNN — Classification

Factor	SVM	KNN
Decision boundary	Learned boundary	Based on neighbours
Training	Higher	Very low
Prediction	Usually faster	Can be expensive
Noise	Can be sensitive to outliers	Sensitive to noisy neighbours
Scalability	Good with suitable implementation	Poorer for very large datasets
Non-linear data	Kernels can help	Naturally handles irregular boundaries

Example: Classifying customers into low-risk and high-risk groups.

SVM finds a hyperplane with the **maximum margin**. KNN finds nearby customers and uses majority voting.

Kernel functions: RBF or polynomial kernels transform the data so that non-linear decision boundaries can be learned.

Conclusion: SVM is usually preferred for high-dimensional classification, while KNN is simpler and useful for smaller datasets.

8. K-Means vs Hierarchical Clustering

Factor	K-Means	Hierarchical
Cluster formation	Iteratively updates centroids	Builds hierarchy
Number of clusters	Usually required beforehand	Can select from dendrogram
Scalability	Good	More expensive
Output	Fixed clusters	Dendrogram + clusters
Interpretability	Moderate	High through dendrogram

K-Means: Assigns points to the nearest centroid and repeatedly updates centroids.

Hierarchical clustering: Starts with individual points and progressively merges clusters.

Cluster shapes: K-Means works best for roughly spherical clusters. Hierarchical clustering can provide more flexible structures depending on linkage.

Conclusion: K-Means is preferable for large datasets, while hierarchical clustering is useful when the underlying cluster hierarchy is important.

9. Genetic Algorithm vs Gradient Descent

Factor	Genetic Algorithm	Gradient Descent
Search	Population-based	Gradient-based
Local minima	Better ability to escape	Can get stuck
Convergence	Usually slower	Usually faster
Derivative required	No	Usually yes
Objective function	Can be non-differentiable	Best for differentiable functions
Computation	Often expensive	Generally efficient

Genetic Algorithm: Uses selection, crossover, and mutation to evolve solutions.

Gradient Descent: Updates parameters in the direction that reduces the objective function.

Example:

GA is suitable for scheduling, combinatorial optimization, and problems without derivatives.

Gradient Descent is highly effective for training linear/logistic regression and neural networks.

Conclusion: Use GA for complex/non-differentiable search spaces and Gradient Descent for smooth differentiable optimization.

10. PCA vs Feature Selection

Factor	PCA	Feature Selection
Method	Creates new features	Selects existing features
Information	Retains maximum variance	May remove some information
Interpretability	Lower	High
Complexity	Can be computationally higher	Often simpler
Features	Transformed components	Original features

PCA: Converts many correlated variables into fewer principal components.

Example:

Feature Selection: Might select only Age, Income, and Credit Score from 20 original features.

Conclusion: PCA is useful when dimensionality is very high and compression is important. Feature selection is preferable when model interpretability is important.

11. Random Forest vs Decision Tree — Credit Risk

Factor	Decision Tree	Random Forest
Number of trees	One	Many
Overfitting	Higher	Lower
Accuracy	Moderate	Usually higher
Interpretability	Very high	Lower
Noise	More sensitive	More robust
Generalization	Can be limited	Usually better

Example: Predicting whether a customer is low-risk or high-risk using income, credit score, debt, and repayment history.

A Decision Tree may overfit training data.

Random Forest creates many trees using random samples and features, then combines their predictions.

Ensemble advantage: Combining many diverse trees reduces variance and improves generalization.

Conclusion: Random Forest generally gives better accuracy and robustness, while a single Decision Tree is easier to explain.

12. Linear vs Polynomial Regression — Housing Prices

Factor	Linear Regression	Polynomial Regression
Relationship	Linear	Non-linear
Complexity	Low	Higher
Overfitting	Lower	Higher
Interpretability	High	Moderate
Flexibility	Limited	High

Linear model:

Polynomial model:

Example:

If house price increases approximately linearly with area, Linear Regression is suitable.

If price changes non-linearly due to location, area, and market effects, Polynomial Regression may perform better.

Conclusion: Linear Regression is simpler and less prone to overfitting. Polynomial Regression captures non-linear relationships but requires careful selection of polynomial degree.

13. K-Means vs DBSCAN — Irregular Clusters

Factor	K-Means	DBSCAN
Cluster shape	Mostly spherical	Arbitrary shapes
Number of clusters	Must specify K	Not required
Noise handling	Poor	Excellent
Parameters	K	ϵ and MinPts
Outliers	Assigned to clusters	Can be labelled noise
Scalability	Very good	Good

Example: GPS locations of vehicles may form irregular road-shaped clusters.

K-Means may incorrectly divide such clusters because it relies on centroids.

DBSCAN groups points based on density and can identify irregularly shaped clusters and outliers.

Conclusion: K-Means is better for large, compact clusters. DBSCAN is better when clusters have irregular shapes and noise is present.

14. RNN vs LSTM — Stock Price Prediction

Factor	RNN	LSTM
Memory	Shorter-term	Long-term
Vanishing gradient	More affected	Reduces problem
Architecture	Simple	More complex
Computation	Lower	Higher

Factor	RNN	LSTM
Long-term dependency	Poorer	Better
Accuracy	Good for short patterns	Often better for long sequences

RNN: Processes sequential data and maintains a hidden state.

Problem: During long sequences, gradients can become very small, causing the network to forget older information.

LSTM: Uses gates—input, forget, and output gates—to control what information is stored and forgotten.

Example: In stock prediction, LSTM can capture relationships between current prices and patterns from much earlier time steps.

Conclusion: RNN is simpler and computationally cheaper, while LSTM is generally more suitable for long-term dependencies.

15. Naïve Bayes vs SVM — Sentiment Analysis

Factor	Naïve Bayes	SVM
Assumption	Conditional independence	No such assumption
Boundary	Probabilistic	Maximum-margin boundary
Training	Very fast	Usually more expensive
High-dimensional data	Excellent	Excellent
Small datasets	Often effective	Can also perform well
Non-linear	Basic NB model is limited	Kernels can model non-linearity

Example: Classifying movie reviews as **Positive** or **Negative**.

Naïve Bayes calculates the probability of each sentiment based on words.

SVM learns a decision boundary that maximizes the margin between positive and negative reviews.

Naïve Bayes is preferable when very fast training and a simple probabilistic model are needed.

SVM is preferable when classification accuracy and a strong decision boundary are more important, particularly with high-dimensional feature vectors.